



What is AWS Lambda?

What is AWS Lambda?

AWS Lambda is a compute service that lets you run code without provisioning or managing servers. Lambda runs your code only when needed and scales automatically, from a few requests per day to thousands per second. You pay only for the compute time that you consume—there is no charge when your code is not running. With Lambda, you can run code for virtually any type of application or backend service, all with zero administration. Lambda runs your code on a high-availability compute infrastructure and performs all of the administration of the compute resources, including server and operating system maintenance, capacity provisioning and automatic scaling, code monitoring and logging. All you need to do is supply your code in one of the languages that Lambda supports.

You can use Lambda to run your code in response to events, such as changes to data in an Amazon Simple Storage Service (Amazon S3) bucket or an Amazon DynamoDB table; to run your code in response to HTTP requests using Amazon API Gateway; or to invoke your code using API calls made using AWS SDKs. With these capabilities, you can use Lambda to build data processing triggers for AWS services such as Amazon S3 and DynamoDB, process streaming data stored in Amazon Kinesis, or create your own backend that operates at AWS scale, performance, and security.

When use AWS Lambda?

AWS Lambda is an ideal compute service for many application scenarios, provided that you can run your application code using the Lambda standard runtime environment and within the resources that Lambda provides.

When using Lambda, you are responsible only for your code. Lambda manages the compute fleet that offers a balance of memory, CPU, network, and other resources. This is in exchange for flexibility, which means you cannot log in to compute instances, or customize the operating system on provided runtimes. These constraints enable Lambda to perform operational and administrative activities on your behalf, including provisioning capacity, monitoring fleet health, applying security patches, deploying your code, and monitoring and logging your Lambda functions.

When use AWS Lambda?

If you need to manage your own compute resources, AWS also offers other compute services to meet your needs. For example:

- Amazon Elastic Compute Cloud (Amazon EC2) offers flexibility and a wide range of EC2 instance types to choose from. It gives you the option to customize operating systems, network and security settings, and the entire software stack. You are responsible for provisioning capacity, monitoring fleet health and performance, and using Availability Zones for fault tolerance.
- AWS Elastic Beanstalk enables you to deploy and scale applications onto Amazon EC2. You retain ownership and full control over the underlying EC2 instances.



Create a Lambda
function with the
console

Create a Lambda function with default function code

In this getting started exercise, you create a Node.js Lambda function using the Lambda console. Lambda automatically creates default code for the function. Next, you manually invoke the Lambda function using sample event data. Lambda runs the function and returns results. You then verify the results, including the logs that your Lambda function created and various Amazon CloudWatch metrics.

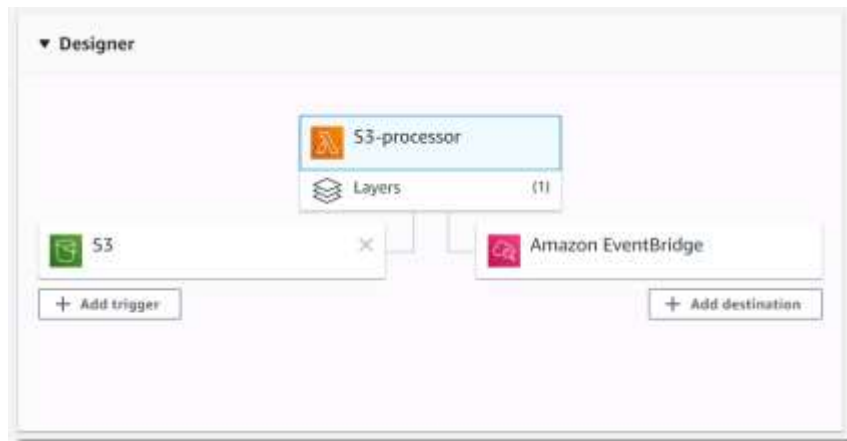
To create a Lambda function with the console

1. Open the Functions page on the Lambda console.
2. Choose **Create function**.
3. Under **Basic information**, do the following:
 - a. For **Function name**, enter **my-function**.
 - b. For **Runtime**, confirm that **Node.js 12.x** is selected.
4. Choose **Create function**.

Use the designer

The **Designer** shows an overview of your function and its upstream and downstream resources. You can use it to configure triggers, layers, and destinations.

Choose **my-function** in the designer to return to the function's code and configuration. For scripting languages, Lambda includes sample code that returns a success response. You can edit your function code with the embedded AWS Cloud9 editor as long as your source code doesn't exceed the 3 MB limit.



Invoke the Lambda function

Invoke your Lambda function using the sample event data provided in the console.

To invoke a function

1. In the upper right corner, choose **Test**.
2. In the **Configure test event** dialog box, choose **Create new test event**. In **Event template**, leave the default **Hello World** option. Enter an **Event name** and note the following sample event template:

```
{  
  "key1": "value1",  
  "key2": "value2",  
  "key3": "value3"  
}
```

You can change keys and values in the sample JSON, but don't change the event structure. If you do change any keys and values, you must update the sample code accordingly.

Invoke the Lambda function

3. Choose **Create**, and then choose **Test**. Each user can create up to 10 test events per function. Those test events are not available to other users.

Lambda runs your function on your behalf. The function handler receives and then processes the sample event.

4. Upon successful completion, view the results in the console.
 - a. The **Execution result** section (below the **Test** button) shows the execution status as **succeeded**. To view the function execution results, expand **Details**. Note that the **logs** link opens the **Log groups** page in the CloudWatch console.
 - b. The **Summary** section shows the key information reported in the **Log output** section (the *REPORT* line in the execution log).
 - c. The **Log output** section shows the log that Lambda generates for each invocation. The function writes these logs to CloudWatch. The Lambda console shows these logs for your convenience. Choose **Click here** to add logs to the CloudWatch log group and open the **Log groups** page in the CloudWatch console.

Invoke the Lambda function

5. Run the function (choose **Test**) a few more times to gather some metrics that you can view in the next step.
6. Near the top of the page, choose the **Monitoring** tab. This page shows graphs for the metrics that Lambda sends to CloudWatch.



Clean up

If you are done working with the example function, delete it. You can also delete the log group that stores the function's logs, and the execution role that the console created.

To delete a Lambda function

1. Open the [Functions page](#) on the Lambda console.
2. Choose a function.
3. Choose **Actions, Delete**.
4. In the **Delete function** dialog box, choose **Delete**.

Clean up

To delete the log group

1. Open the Log groups page of the CloudWatch console.
2. Select the function's log group (`/aws/lambda/my-function`).
3. Choose **Actions, Delete log group(s)**.
4. In the **Delete log group(s)** dialog box, choose **Delete**.

To delete the execution role

1. Open the Roles page of the AWS Identity and Access Management (IAM) console.
2. Select the function's role (`my-function-role-31exxmpl`).
3. Choose **Delete role**.
4. In the **Delete role** dialog box, choose **Yes, delete**.

You can automate the creation and cleanup of functions, log groups, and roles with AWS CloudFormation and the AWS Command Line Interface (AWS CLI).

Create a function defined as a container image

Prerequisites

To complete the following steps, you need a command line terminal or shell to run commands. Commands are shown in listings preceded by a prompt symbol (\$) and the name of the current directory, when appropriate:

```
~/lambda-project$ this is a command  
this is output
```

For long commands, an escape character (\) is used to split a command over multiple lines.

On Linux and macOS, use your preferred shell and package manager.

Create a function defined as a container image

Create the container image

AWS provides a set of base images in the Amazon Elastic Container Registry (Amazon ECR). In this getting started exercise, we use the Node.js base image to create a container image.

In the following commands, replace `123456789012` with your AWS account ID.

Create a function defined as a container image

To create an image using the AWS Node.js 12 base image

1. On your local machine, create a project directory for your new function.
2. Create a file named `app.js` in your project directory. Add the following code to `app.js`:

```
exports.handler = async (event) => {  
  // TODO implement  
  const response = {  
    statusCode: 200,  
    body: JSON.stringify('Hello from Lambda!'),  
  };  
  return response;  
};
```

Create a function defined as a container image

3. Use a text editor to create a new file named `Dockerfile` in your project directory. Add the following content to `Dockerfile`:

```
FROM public.ecr.aws/lambda/nodejs:12

# Copy function code and package.json
COPY app.js package.json /var/task/

# Install NPM dependencies for function
RUN npm install

# Set the CMD to your handler
CMD [ "app.handler" ]
```

4. Create the `package.json` file. From your project directory, run the `npm init` command. Accept all of the default values:

```
npm init
```


Create a function defined as a container image

5. Build your Docker image. From your project directory, run the following command:

```
docker build -t hello-world .
```

6. (Optional) AWS base images include the Lambda runtime interface emulator, so you can test your function locally.

- a. Run your Docker image. From your project directory, run the `docker run` command:

```
docker run -p 9000:8080 hello-world:latest
```

- a. Test your Lambda function. In a new terminal window, run a curl command to invoke your function:

```
curl -XPOST "http://localhost:9000/2015-03-31/functions/function/invocations" -d '{}'
```

Create a function defined as a container image

Upload the image to the Amazon ECR repository

1. Authenticate the Docker CLI to your Amazon ECR registry.

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com
```

2. Create a repository in Amazon ECR using the `create-repository` command.

```
aws ecr create-repository --repository-name hello-world --image-scanning-configuration scanOnPush=true --image-tag-mutability MUTABLE
```

3. Tag your image to match your repository name using the `docker tag` command.

```
docker tag hello-world:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/hello-world:latest
```

4. Deploy the image to Amazon ECR using the `docker push` command.

```
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/hello-world:latest
```

Create a function defined as a container image

Update the user permissions

Make sure that the permissions for the IAM user or role that creates the function contain the AWS managed policies `GetRepositoryPolicy` and `SetRepositoryPolicy`.

For example, use the IAM console to create a role with the following policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": ["ecr:SetRepositoryPolicy", "ecr:GetRepositoryPolicy"],
      "Resource": "arn:aws:ecr:<region>:<account>:repository/<repo name>/"
    }
  ]
}
```

Create a function defined as a container image

Create a Lambda function defined as a container image

Use the Lambda console to create a function defined as a container image.

To create the function with the console

1. Open the Functions page on the Lambda console.
2. Choose **Create function**.
3. Choose the **Container image** option.
4. Under **Basic information**, do the following:
 - a. For **Function name**, enter **my-function**.
 - b. For **Container image URI**, enter the URI of the Amazon ECR image that you created previously.
5. Choose **Create function**.

Lambda creates your function and an execution role that grants the function permission to upload logs. Lambda assumes the execution role when you invoke your function, and uses the execution role to create credentials for the AWS SDK and to read data from event sources.

Create a function defined as a container image

Invoke the Lambda function

Invoke your Lambda function using the sample event data provided in the console.

To invoke a function

1. In the upper right corner, choose **Test**.
2. In the **Configure test event** dialog box, choose **Create new test event**. In **Event template**, leave the default **Hello World** option. Enter an **Event name** and note the following sample event template:

```
{
  "key1": "value1",
  "key2": "value2",
  "key3": "value3"
}
```

Create a function defined as a container image

3. Choose **Create**, and then choose **Test**. Each user can create up to 10 test events per function. Those test events are not available to other users.
Lambda runs your function on your behalf. The function handler receives and then processes the sample event.
4. Upon successful completion, view the results in the console.
 - a. The **Execution result** section (below the **Test** button) shows the execution status as **succeeded**. To view the function execution results, expand **Details**. Note that the **logs** link opens the **Log groups** page in the CloudWatch console.
 - b. The **Summary** section shows the key information reported in the **Log output** section (the *REPORT* line in the execution log).
 - c. The **Log output** section shows the log that Lambda generates for each invocation. The function writes these logs to CloudWatch. The Lambda console shows these logs for your convenience. Choose **Click here** to add logs to the CloudWatch log group and open the **Log groups** page in the CloudWatch console.

Create a function defined as a container image

- Run the function (choose **Test**) a few more times to gather some metrics that you can view in the next step.
- Near the top of the page, choose the **Monitoring** tab. This page shows graphs for the metrics that Lambda sends to CloudWatch.



Create a function defined as a container image

Clean up

If you are done working with your function, delete it. You can also delete the log group that stores the function's logs and the execution role that the console created.

To delete a Lambda function

1. Open the Functions page on the Lambda console.
2. Choose a function.
3. Choose **Actions, Delete**.
4. In the **Delete function** dialog box, choose **Delete**.

Create a function defined as a container image

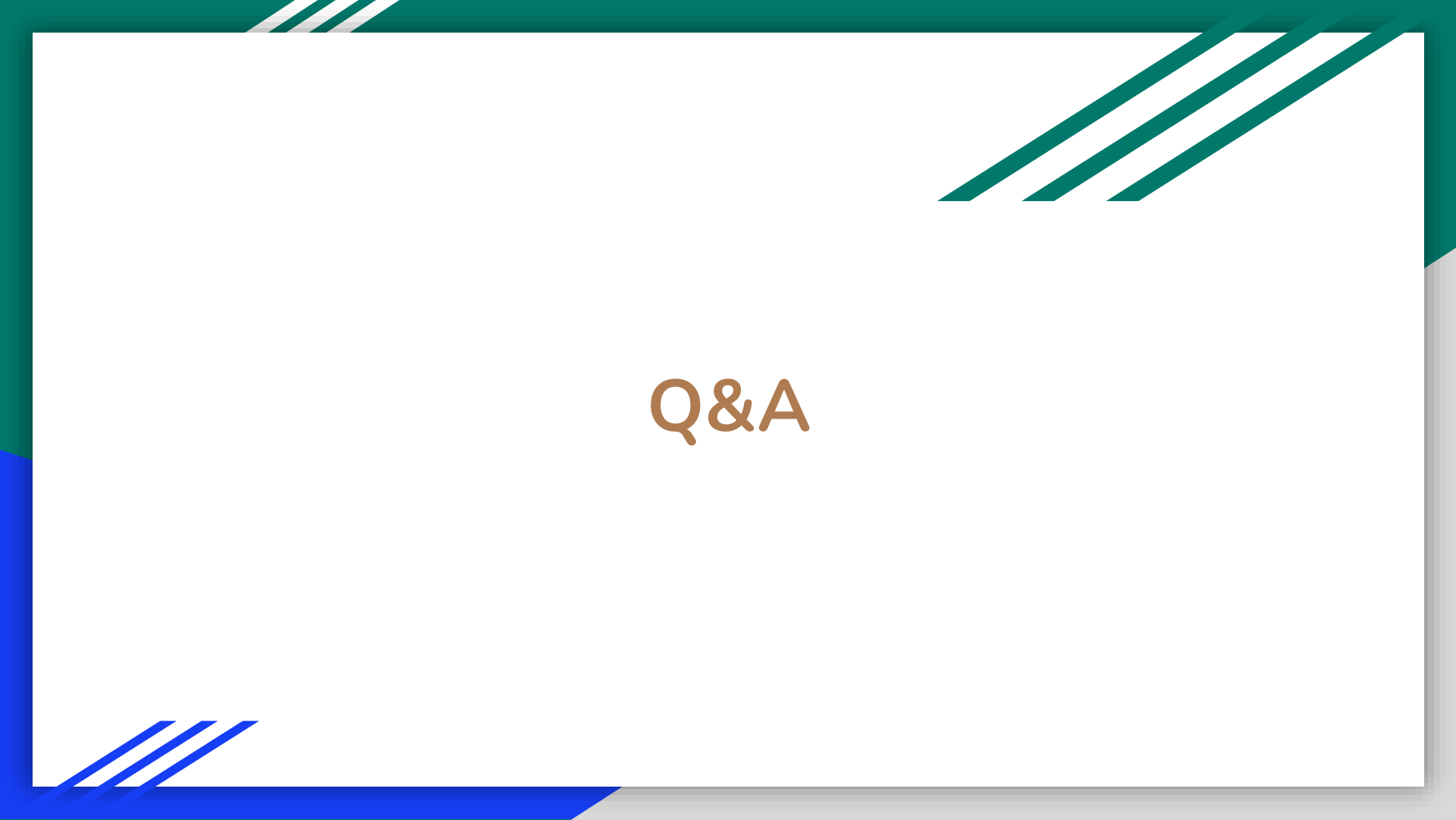
To delete the log group

1. Open the Log groups page of the CloudWatch console.
2. Select the function's log group (/aws/lambda/my-function).
3. Choose **Actions, Delete log group(s)**.
4. In the **Delete log group(s)** dialog box, choose **Delete**.

To delete the execution role

1. Open the Roles page of the IAM console.
2. Select the function's role (my-function-role-31exxmpl).
3. Choose **Delete role**.
4. In the **Delete role** dialog box, choose **Yes, delete**.

You can automate the creation and cleanup of functions, log groups, and roles with AWS CloudFormation and the AWS CLI.



Q&A